Part I – Classical Neural Machine Translation

1. Objective

In this section, we have tried design and implement a complete Neural Machine Translation(NMT) framework for Hindi ↔ Marathi translation using recurrent neural architectures.

2. Task Description

We have built a Seq2Seq translation framework consisting of an LSTM encoder, an LSTM decoder, with an Attention mechanism.

3. Dataset

We have used the dataset provided by the assignment. The train set containing 2,41,000+ Hindi and Marathi sentence pairs, have been used for training of the frameworks.

4. Initial Embedding

Two different types of embeddings were used for experimentation on the architecture. One is random embedding for both Hindi and Marathi. Second one is BERT embeddings for both the languages.

5. Experimental Setup

As we lack computational resources, the architecture parameters have been reduced, and run for a small number of epochs. The machine has an Intel core i7 processor with 64 gb RAM. We used an Nvidia T1000 GPU with 4gb dedicated vRAM.

6. Preprocessing and Tokenization

No preprocessing done for this. SentencePiece tokenizers were trained and used for Hindi and Marathi. The first 2,00,000 pairs of train sets are used for training, and the rest are used for validation. The final vocabulary for the languages are 28,565 for Hindi and 29,982 for Marathi.

7. Architectural Design

We have used an encoder decoder architecture for machine translation inspired by Bahdanau et al (https://doi.org/10.48550/arXiv.1409.0473). They have used GRU for translation, but as per the requirement of the assignment, we have used LSTM for both encoder and decoder. Bahdanau et al mentions that the GRU can be replaced with LSTM for translation tasks.

7.a Encoder:

Embedding dimension - 128 (Random Initialization) and 768 (BERT Embedding)

Hidden state dimension - 256 * 2 (multiplied by 2 because of bi-LSTM)

Dropout - 0.3

7.b Decoder:

Embedding dimension - 128 (Random Initialization) and 768 (BERT Embedding)

Hidden state dimension - 256

Dropout - 0.2

As the encoder and decoder hidden state sizes are different, the state is not directly passed from encoder to decoder, it is passed through a simple Artificial Neural Network.

7.c Attention

Bahdanau attention was used at the decoder side to make better use of the inputs while on the decoder side.

$$e_{t,i} = a(\mathbf{s}_{t-1}, \mathbf{h}_i)$$

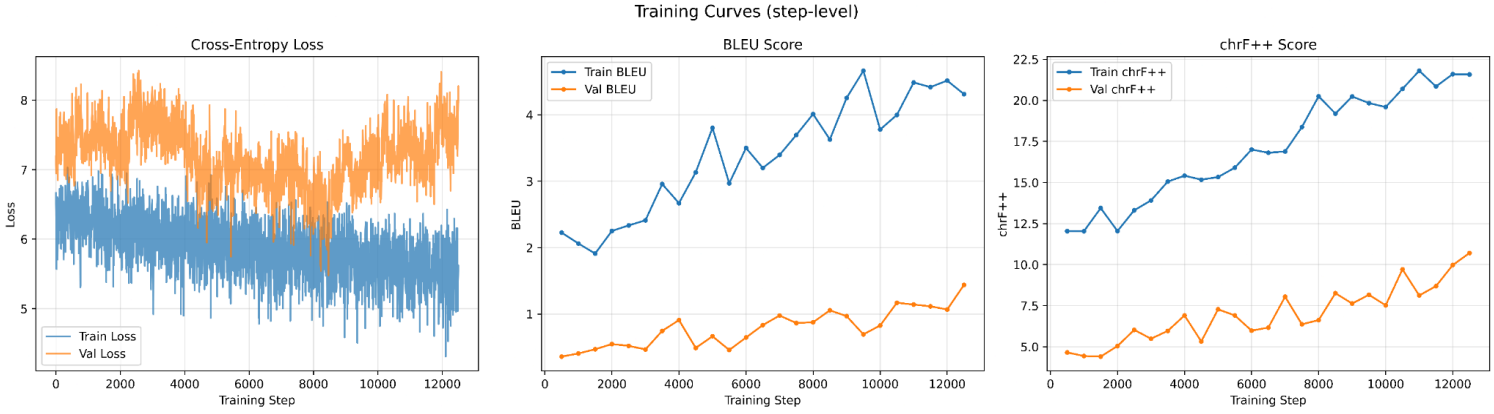
$$a(\mathbf{s}_{t-1}, \mathbf{h}_i) = \mathbf{v}^T \tanh(\mathbf{W}[\mathbf{h}_i \; ; \; \mathbf{s}_{t-1}])$$

$$\alpha_{t,i} = \text{softmax}(e_{t,i})$$

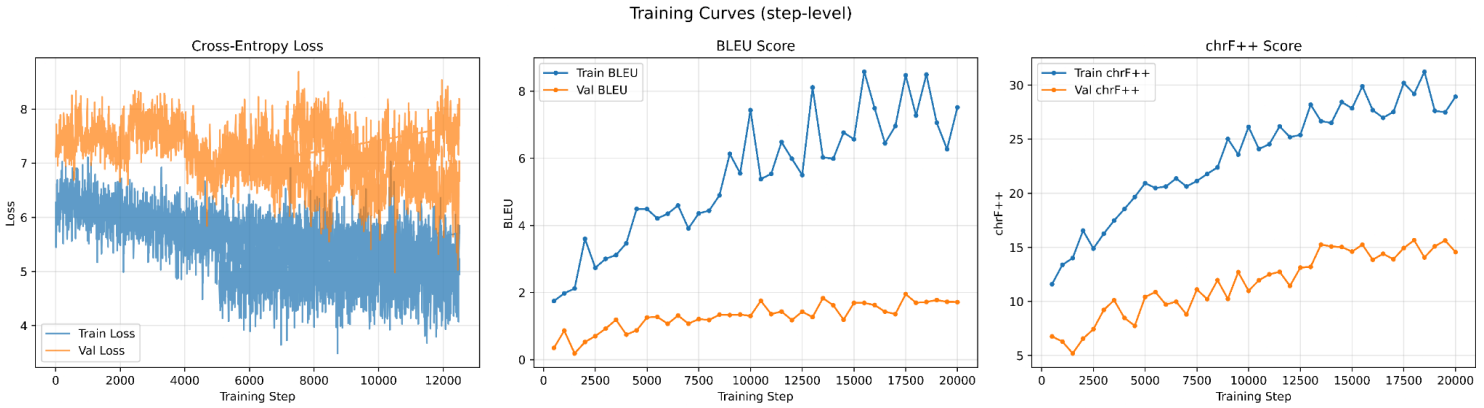
Results:

For this assignment, two different versions were asked for. First with random embedding (Experiment 1.1) and second one with BERT embedding (Experiment 1.2). For evaluation, cross-entropy loss, BLEU-100, and CHRF++100 are used.

BLEU focuses on n-gram similarity between model output and ground truth, whereas CHRF++100 focuses on their character level similarity.
The experiment 1 model has 36,426,398 trainable parameters, the experiment 2 model has 95,051,038 trainable parameters. The difference in the number of parameters comes from the embeddings. As in experiment 1, random initialization was used, so the embeddings were learnable, whereas in bert based initialization, we have frozen the embedding part.



Experiment 1.1 Result



Experiment 1.2 Result

The X-axis in the plots does not show epoch count, they are step count. Due to lack of computational resources and time, we have used step-wise evaluation. In this, instead of calculating loss, and scores every epoch, we calculate them after every few batches of training, these are called steps. In our experiment, step length is 5.
In both random and bert based initializations of embeddings, the losses slowly decrease, but in the bert based initialization, the rate of improvement is slightly better. The

training score is normally better than the validation score, but both have similar trends of improvement. The overall score is not that good, but the plot shows a trend of improvement, so we can say that with more epochs of training and more time, the scores will improve.
The result on test data:
Experiment 1.1-

| Test BLEU: 3.96 | Test chrF++: 25.36 |

Experiment 1.2-

| Test BLEU: 8.58 | Test chrF++: 36.96 |

We can see a significant difference in the result of bert based embedding initialization. We can conclude that bert embeddings produce better results than random initialization.

Failure Analysis:

- BLEU 100 and CHRF++100 both has a range of 1-100, so our score is not even nearly good. We assume this might be because the training time is too less. With more training, this should get better.
- There is a large gap between result of validation and testing. This might have been caused by a bad choice of validation set.
- The losses and the scores fluctuate a lot. This might have been the cause of bad choice of learning rare, and low training time. We can make better learning rate schedulers for better and smooth results.

Part II – Language Model Pretraining and Translation

BERT : BERT encoder model is trained from scratch for hindi language understanding.

Experimental Setup

As we lack computational resources, the architecture parameters have been reduced, and run for a small number of epochs. The machine has an Intel core i7 processor with 64 gb RAM. We used an Nvidia T1000 GPU with 4gb dedicated vRAM. Python 3.10 was used as a programming language.

DATASET :

The dataset used for training the model to build up the understanding of Hindi language was taken from AI4Bharat Sangraha (verified versions), a subset, for Hindi language (0.25M) from the huge corpus of Indian language data.

Data Preprocessing :

- Remove English alphabet characters.
- Remove numeric values.
- Normalize white space.
- Remove short sequences.

Tokenization :

For the tokenization, Sentence Piece Tokenizer was used to convert a subword tokenization algorithm that converts raw text into smaller units (tokens) before feeding them into a Transformer. Unlike traditional tokenizers that split on spaces first, SentencePiece treats the input as a sequence of Unicode characters and learns token boundaries automatically.

text → tokens → IDs

Consider a sentence :
मेरा नाम सलोनी है

Possible tokenization :
[“_मेरा”, “__ नाम” ,”__ सलोनी” , “__ है”]

IDs :
[256, 18, 56,32]
The table below shows the configurations, where, UNK is “unknown”, PAD is “Padding”, BOS is "Beginning of the sentence” and EOS is “Ending of the sentence”.

Configuration :

Parameters	**Value**
Vocabulary Size	20,0000
Model Type	BPE
Character Coverage	0.9995
PAD	0
UNK	1
BOS	2
EOS	3
CLS	4
SEP	5
MASK	6

Sequence Formatting

Each sequence becomes:

[CLS] token1 token2 ... tokenN [SEP] [PAD]

Data Preparation :

The diagram below shows the data preparation pipeline for Hindi text before passing it to the BERT encoder.The process starts with Hindi Text, which is the raw dataset containing sentences written in Hindi. Since neural networks cannot directly process text, the next step is SentencePiece Encoding.After tokenization, the encoded text goes through Sequence Packing. Here, tokenized sentences are arranged into fixed-length sequences suitable for model training. Instead of keeping every sentence separately, multiple shorter sequences may be combined efficiently to reduce unused space and improve GPU utilization.The packed sequences are then passed to Padding. Since deep learning models process batches of equal-sized inputs, shorter sequences are padded with a special token (such as [PAD]) until they reach the required length. This ensures consistent tensor dimensions during training.Once preprocessing is complete, the dataset is divided into **Train / Validation Split**. The training set is used to learn model parameters, while the validation set is used to monitor performance and prevent overfitting.

Finally, the processed data is stored as **NumPy Shards**. Instead of saving the entire dataset in one large file, it is split into multiple .npy chunks (shards). Sharding allows faster loading, reduced memory usage, and efficient parallel training during large-scale model pretraining.

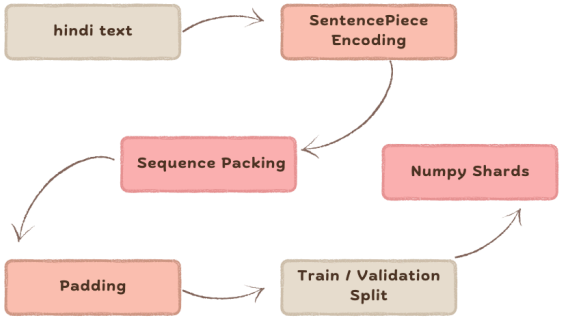
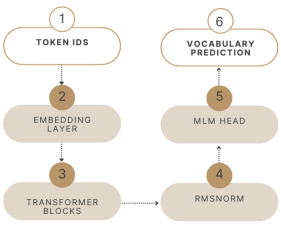


Table below shows the configurations:

Parameter	**Value**
Validation Split	10%
Sequence Length	128
Shard Size	10M

Model Architecture :

The diagram shows the **forward pass of the Hindi BERT-style Masked Language Model (MLM)** during pretraining.



1. Token IDs (Input):

This is the text after **SentencePiece tokenization** which is later converted to IDs.

Output shape: (B,T)

(batch size × sequence length) where total_batch_size =2048

Example:(4,128)

2. Embedding Layer:

The model cannot understand integers directly.Embedding converts each token ID into a dense vector.The embeddings are randomly initialized.

For example :

145 → [0.21,-0.52,0.13,...]

87 → [0.89,0.33,-0.14,...]

The embedding size for the BERT model is:768

then:

(B,T)→(B,T,768)

Meaning:

(4,128)→(4,128,768)

Now every word becomes a semantic representation.

3.Transformer Blocks :

This is the main learning component.Each Transformer block contains:

Attention → Residual Connection → Normalization→Feed Forward Network

Attention :

Positional embedding is used inside attention because the Transformer architecture processes all tokens in parallel and does not inherently understand word order. In our architecture, instead of positional embedding, RoPe (Rotary Positional Embedding) is used. RoPE rotates the Query (Q) and Key (K) vectors based on token position before computing attention.

The attention becomes:

$$Attention(Q, K, V) = softmax\left(\frac{Q_{rot}K_{rot}^{T}}{\sqrt{d_k}}\right)V$$

where:

$$Q_{rot} = R(m)Q, \qquad K_{rot} = R(n)K$$

- $m, n \rightarrow$ positions of tokens
- $R(\cdot) \rightarrow$ rotation matrix

Group Query Attention : Instead of multi head attention, group query attention is used. Grouped Query Attention (GQA) is an optimization of standard multi-head attention that reduces memory and speeds up inference while keeping most of the model quality.

In Standard Multi-Head Attention (MHA) every attention head has its own Query, Key, and Value projections. This gives good performance but requires storing many K and V tensors, whereas , GQA keeps many Query heads but shares fewer Key and Value heads.Each group of query heads shares the same K and V.

Attention becomes:

$$Attention(Q_i, K_g, V_g)$$

where:

- Q_i → individual query head
- K_g, V_g → shared key/value group

The table below shows the parameters used in GQA :

Hyperparameter	**Value**
Layers	4
Hidden size	768
Attention Heads	4
KV heads	2
Vocabulary	20,000
Sequence Length	128

Residual Connection:

Residual connection (skip connection) is added in attention layers to make deep Transformer networks easier to train and preserve information from earlier layers.

Normalization: Normalization is required to properly scale the input vectors. Here Root Mean Square Normalization or RMSNorm has been used.

Formula:

$$RMSNorm(x) = x\frac{1}{\sqrt{mean(x^2) + \epsilon}}$$

Feed Forward Network:

A simple feed forward neural network is used after the residual connection to adjust the dimension of the input, otherwise constantly adding residual connection will increase vector size.

4. Masked Language Model :

Training strategy used for BERT-style Masked Language Modeling (MLM) with dynamic masking.The objective of Masked Language Modeling is to train the model to predict missing words from context. Instead of showing the complete sentence, some tokens are hidden or modified, and the model learns to recover the original token.

Dynamic Masking :

Dynamic masking means **mask positions are generated during training**, not fixed beforehand.

Epoch 1:

राम [MASK] गया

Epoch 2:

[MASK] स्कूल गया

Epoch 3:

राम स्कूल [MASK]

This exposes the model to more prediction patterns and improves generalization.

The 80–10–10 Rule

After selecting tokens for masking, BERT does **not always replace them with `[MASK]`**.

80% → Replace with `[MASK]`

Example:

मैं खाना खाता हूँ

↓

मैं [MASK] खाता हूँ

Model predicts: खाना

This teaches missing-token prediction.

10% → Replace with random token

Example:

मैं किताब पढ़ता हूँ

↓

मैं आम पढ़ता हूँ

Target still: किताब

This prevents the model from depending too much on the `[MASK]` symbol.

10% → Keep unchanged

Example:

मैं किताब पढ़ता हूँ

Target: किताब

The word remains visible, but the model still tries to predict it.This helps reduce mismatch because during inference there will be **no `[MASK]` tokens**.

The **Mask Ratio** was kept **15%** that is Only 15% of tokens in each sequence are chosen for prediction.

The table below represents the optimization specifications that was used in the architecture :

parameter	**Value**
Optimizer	AdamW
Max Learning Rate	1e-4
Min Learning Rate	1e-5
Scheduler	Cosine Decay
Warmup	1000
Max Steps	20,000

The model training uses the AdamW optimizer together with a Cosine Learning Rate Scheduler and Linear Warmup. These settings are selected to achieve stable optimization and efficient convergence during BERT pretraining.

This strategy consists of two phases:

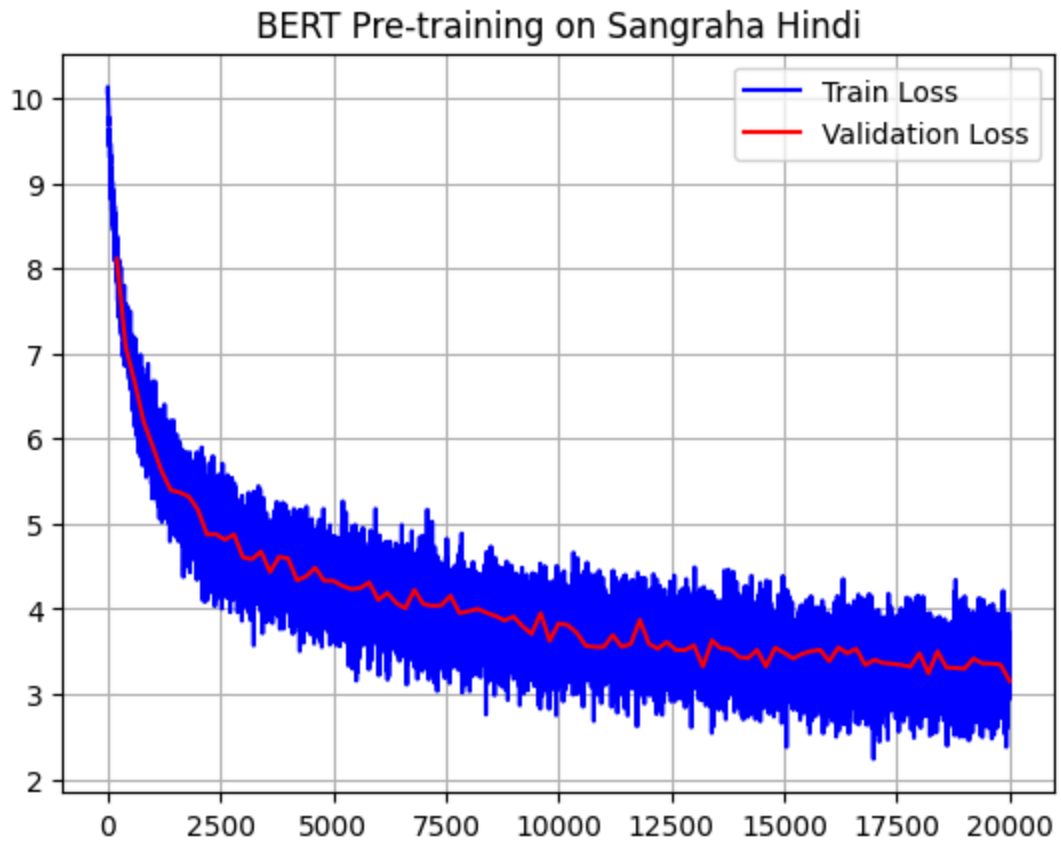
1. **Linear Warmup**
2. **Cosine Decay**

The table below shows the parameters of batching :

Parameters	Value
Global Batch	2048
Micro Batch	4
Sequence Length	128

Gradient accumulation is used.

Result:
The figure given below represents the train loss vs validation loss of BERT training from scratch :



The figure clearly depicts that training as well as validation loss is decreasing over the period of time and didn't attend saturation yet cause number of epochs was kept 2 due to computational constraints. If the resource would have been better the result can potentially improve a lot.

GPT-2 : GPT decoder model is trained from scratch for Marathi language understanding.

DATASET :

The dataset used for training the model to build up the understanding of Hindi language was taken from AI4Bharat Sangraha (verified versions), a subset, for Marathi language (1M) from the huge corpus of Indian language data.

The experimental setup, data preparation and preprocessing, hyperparameter , GQA, RMSNorm, RoPE etc. were kept the same as the BERT model, only the difference is in the Masking.
GPT is an autoregressive model that means GPT generates text one token at a time, where each new token depends only on previously generated tokens.

This means :

$$P(sentence) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \dots$$

The model predicts the **next token given all previous tokens**.

Example

Input:

भारत की राजधानी

GPT predicts:

Step 1:

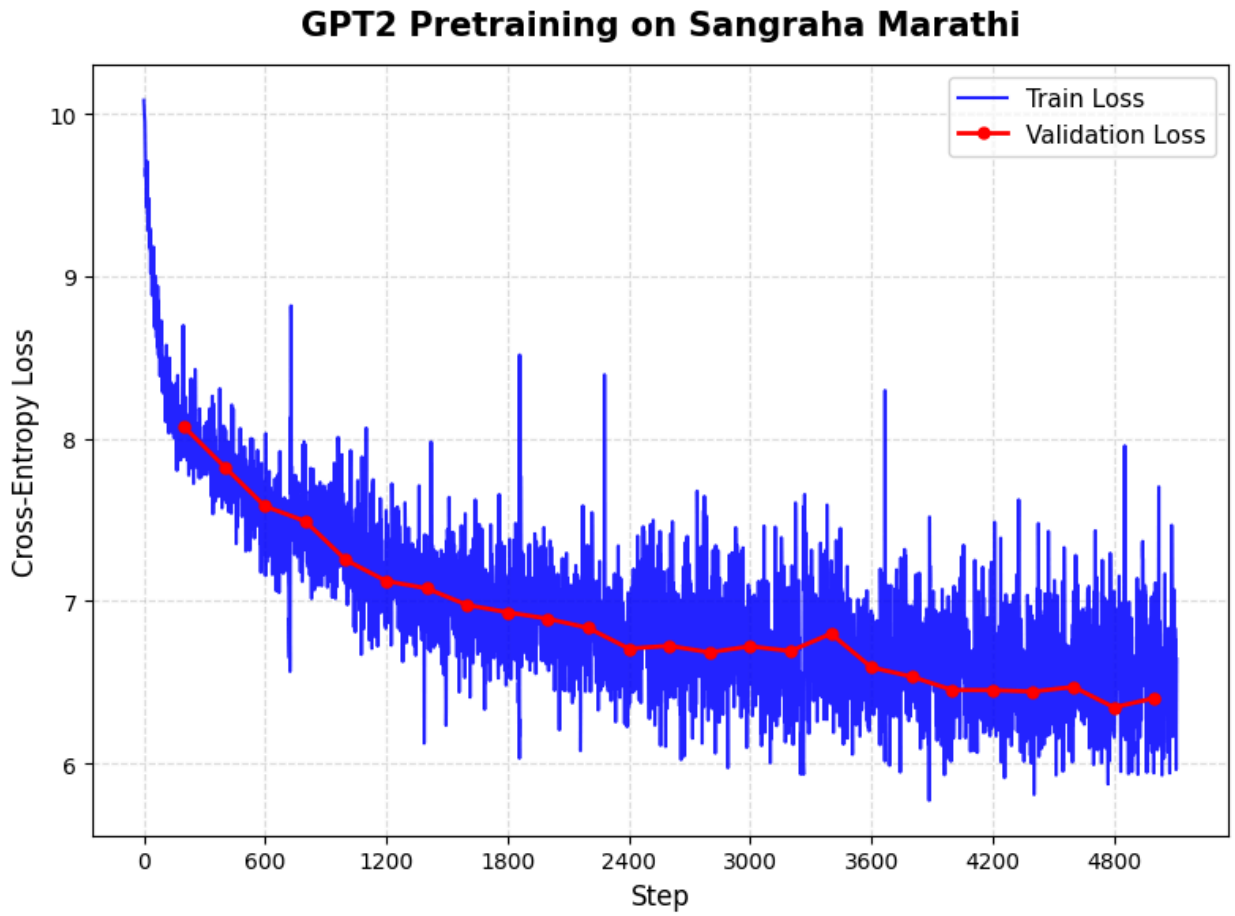
भारत की राजधानी → दिल्ली

Step 2:
भारत की राजधानी दिल्ली → है

Final output:
भारत की राजधानी दिल्ली है।

⇒ GPT enforces Causal Masking .

Result :
The figure given below represents the train loss vs validation loss of BERT training from scratch :



Here also we can see that training as well as validation loss is decreasing over the period of time and didn't attend saturation yet because the number of epochs was kept 2 due to computational constraints. If the resource would have been better the result can potentially improve a lot.

Hindi ⇒Marathi Neural Machine Translation:

Experimental Setup:

For the final stage fine-tuning, we have used a kaggle workspace. It has 30 gb RAM, 2 T4 GPU with 15 gb vRAM each. We have used Python 3.12.12 as a programming language.

Model Training:

For Hindi to Marathi translation , BERT model and GPT 2 model pretrained from scratch using AI4Bharat Sangraha (verified versions) data is used.Then, they are combined for translation using cross-attention and after that fine tuned on the data provided.

Cross-Attention : Cross-attention is the mechanism that allows the decoder to look at the encoder output while generating translations.

In this model:

- The encoder processes the Hindi sentence.
- The decoder generates the Marathi sentence.
- Cross-attention acts as the bridge between them.

The idea is:

- Query (Q) comes from decoder and tells What information is needed now
- Key (K) and Value (V) come from encoder outputs where Key(K) represents source words and Value (V) represents actual information passed to the decoder.

The attention scores are computed using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

This computes similarity between:

- the current decoder token representation,
- and all encoder token representations.

The decoder then focuses more strongly on relevant Hindi words while generating each Marathi word.

For example:

Hindi input:
मैं स्कूल जाता हूँ
While generating Marathi word:
शाळेत
the decoder attends strongly to the Hindi word:
स्कूल
Thus , cross-attention performs **soft alignment** between source and target languages.

Padding Mask in Cross-Attention :

The model also masks padding tokens.This ensures that:

- PAD tokens in the Hindi sentence receive near-zero attention,
- Only real words contribute to translation.

The role of Cross-Attention in this architecture is that Cross-attention is the newly added component that connects both models.

⇒The weights of the Cross-attention are new and randomly initialized.

Teacher Forcing :

Teacher forcing is a training technique used in sequence-to-sequence models where the decoder is fed the actual target token from the dataset instead of its own previously generated prediction.

In this implementation, teacher forcing is created during dataset preparation itself.

Decoder input : BOS + tokens

Decoder targets: tokens + EOS

Suppose the Marathi sentence is:

मी शाळेत जातो

After tokenization:

[मी, शाळेत, जातो]

Then:

Decoder Input : [BOS, मी, शाळेत, जातो]

Target Output : [मी, शाळेत, जातो, EOS]

So during training:

- At time step 1, decoder receives BOS and predicts मी
- At time step 2, decoder receives the **correct token** मी and predicts शाळेत
- At time step 3, decoder receives शाळेत and predicts जातो

Thus, the decoder always sees the correct previous token during training.Without teacher forcing, the decoder would feed its own predicted token back as the next input, causing errors to accumulate rapidly during training.The decoder input is therefore called a shifted-right target sequence.

Beam Search Decoding :

Beam search is a decoding strategy used during inference (translation generation) to produce better translations than greedy decoding.Instead of selecting only the single most probable token at every step, beam search keeps multiple candidate

sequences and explores several possible translations simultaneously. Beam search keeps the top **k** candidate sequences at each step. Here:

- k = beam width (4 in our code)
- multiple hypotheses are maintained simultaneously.

For beam width = 3:

Candidate	**Probabillity**
मी शाळेत	0.78
मी घरी	0.56
आम्ही शाळेत	0.30

At the next step, all candidates are expanded again and rescored.This allows the model to recover from locally suboptimal choices and generate more fluent translations.Each beam stores, token sequence which is partially generated sentence and score which is cumulative log probability.

At every decoding step:

1. Each candidate sequence is fed into the decoder.
2. The model predicts probabilities for the next token.
3. Top-k next tokens are selected.
4. New candidate sequences are formed.

Model Training :

Two-Phase Training Strategy is used :

Phase 1: Decoder + Cross-Attention only (encoder frozen): The encoder produces stable Hindi representations; only the decoder and the newly-initialized

cross-attention weights are updated. This is fast and avoids catastrophic forgetting.

Phase 2: Joint fine-tuning (encoder unfrozen): Use a much lower learning rate for the encoder (10× smaller) so it adapts toward translation without losing its Hindi language understanding.

⇒Number of epochs is kept 2 due to computational constraints.

Results:

BLEU 100 (left): focuses on exact n-gram overlap with reference translations.

ChrF++100 (right): character-level metric, usually smoother and more sensitive to morphology/partial matches.

BLEU Score Analysis:

BLEU fluctuates between roughly:

- Train: 4.0–8.1
- Test: 3.0–7.9

Peaks occur around:

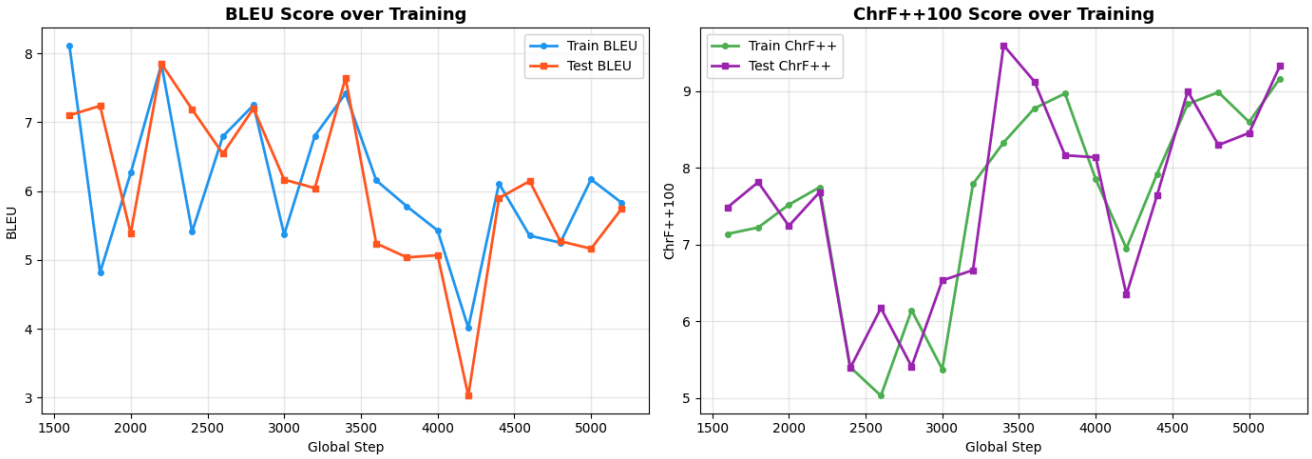
- step ~2200
- step ~3400

Significant drop around:

- step ~4200

The curves overlap a lot,showing the model is not memorizing training data excessively due to the number of epochs being 2 as well as the number are parameter less due to computational constraint and the improvements transfer to unseen data.

The curve of train and test data are close showing the descent generalization.



ChrF++100 Analysis :

- Scores improve overall:from ~7.1 → ~9.2
- Test and train remain close.
- Less noisy than BLEU.

Unlike BLEU, ChrF++ shows:

- more consistent learning
- gradual quality improvement

This suggests:

- translations are improving at the character/subword level
- the model is learning morphology and partial word correctness

Highest ChrF++ occurs near:

- step ~5200

Failure Analysis :

- In BLEU100 plot the curves overlap a lot, showing the model is not memorizing training data excessively due to the number of epochs being 2 as well as the number are parameter less due to computational constraint.
- Test BLEU falls to about 3.0,Since it recovers afterward, it’s probably not permanent divergence.
- In CHRF++100,Test scores sometimes exceed train scores This can happen because the test set may be easier or sampling variance .